Proof of concept.

Testing the energy-aware runtime on a laptop battery, before any solar hardware is bought

SCHEDULE	
TESTS	The WP2 runtime: battery ladder, surplus window and safe stops
HARDWARE	One 16 GB laptop. No solar and no new purchases.
STAND-INS	The laptop battery stands in for the LiFePO4 battery, and the charger for solar surplus
EFFORT	About two days of work
OUTPUTS	Public code repository with a README to reproduce the test, and the measured log and chart
LICENCES	Code Apache 2.0 · logs and chart CC BY 4.0
STATUS	Demonstration complete, 27 October 2026. All five pass criteria met.

Why test this first

The proposal rests on one idea: a node that runs short of energy should step down to a smaller model instead of switching off, and it should never lose work when the power goes. That logic does not need solar panels to test. A laptop already has a battery it can read, a charger that comes and goes, and enough memory to run three sizes of an open model.

Testing on a laptop first shows that the runtime's decisions work, and turns the proposal's thresholds into measured behaviour before grant money is spent on hardware.

What it tests

1 Battery ladder

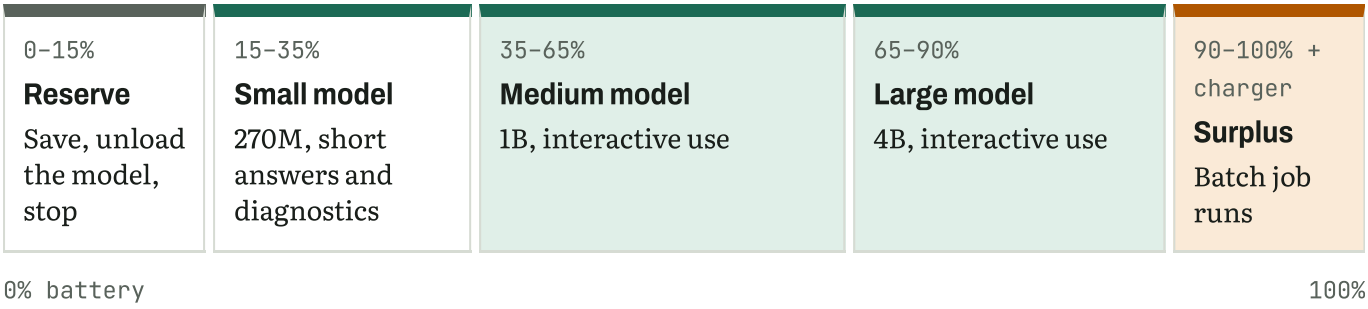
As charge falls, the runtime unloads the larger model and loads a smaller one, using the same bands as the proposal.

2 Surplus window

A deferrable batch job runs only while the charger is connected and the battery is at 90% or more. It pauses as soon as the charger is pulled and resumes when it is reconnected.

3 Safe stops

Each finished result is written to a temporary file and then renamed, so an abrupt stop leaves either the whole result or none of it. On restart, the runtime skips finished work. At 15% it unloads the model and stops cleanly.



The same bands as the full node's ladder, with models small enough to run on a laptop's processor. The charger stands in for the sun.

FULL NODE IN THE PROPOSAL	PROOF OF CONCEPT
LiFePO4 battery, read from the charge controller	Laptop battery, read from the operating system every 10 seconds
Solar surplus: battery near full and strong sun	Charger connected and battery at 90% or more
Model ladder of about 4B, 20B and 120B parameters	270M, 1B and 4B models from one open family (Gemma 3), small enough to run on the laptop's processor
Batch work: transcription, document processing, evaluations	Summarising a public-domain book, one chunk at a time
Unplanned power loss	Ending the process abruptly in the middle of a chunk
Power logging from the charge controller	The laptop's own reported battery discharge rate

How the runtime works

A small Python program sits between the laptop's battery and a local model server (Ollama). Every 10 seconds it:

1. Reads the charge level and whether the charger is connected.
2. Picks the model for that charge band, unloading the previous one to free memory.
3. Runs the next chunk of the batch job if it is in the surplus window, or pauses the job if not.
4. Every five minutes, sends a short test prompt to the current model and records its speed.
5. At the reserve band, unloads the model and stops.
6. Writes every event to a log: time, charge, charger state, power draw, model, event, progress and speed.

Test plan

Demonstration

STEP	ACTION	EXPECTED RESULT
1	Charger connected, battery nearly full. Start the runtime.	The large model loads. The batch job starts and chunks complete one by one.
2	End the process in the middle of a chunk, then restart it.	The runtime resumes at the first unfinished chunk. No result is lost, duplicated or half-written.
3	Pull the charger.	The batch job pauses after the current chunk. Test prompts are still answered.
4	Reconnect the charger.	The batch job resumes.
5	Disconnect and leave it running.	The model steps down at each band, then the runtime unloads it and stops at reserve.

For the demonstration, the bands are narrowed to 96%, 93% and 90%, the batch job runs from 97%, and a test prompt goes out every 30 seconds, so every step happens within minutes. The abrupt stop comes before the charger is pulled, because the batch job only runs while the charger is connected.

Pass criteria

CHECK	PASS IF
Ladder steps	All three transitions happen, each within one 10-second check of the charge crossing its band
Surplus window	The batch job pauses within one chunk of the charger being pulled, and resumes when it is reconnected
Abrupt stops	Zero chunks lost, duplicated or half-written, across every restart
Reserve	The model is unloaded and the runtime stops once charge falls into the reserve band
Measurements	Speed and average power draw reported for each model. No target is set.

These are the proposal's success measures for safe stops and the battery ladder, at laptop scale. Any check that fails is reported as it happened.

What it does not show

The proof of concept tests the runtime's decisions, not the energy system. It does not show:

- Solar generation, weather forecasting, or a LiFePO4 battery. A laptop's charge level is an estimate made by its own firmware.
- The 128 GB node, its 120 W load, or heat in a real enclosure.
- The pool, the gateway or the batch API.
- Setup by people without a systems background.
- Metered power. The laptop's power readings are averaged by its firmware and do not replace a meter.

These are what the grant would fund: WP1 for the hardware, WP2 for forecasting, WP3 for the pool and WP4 for setup by non-experts.

Results

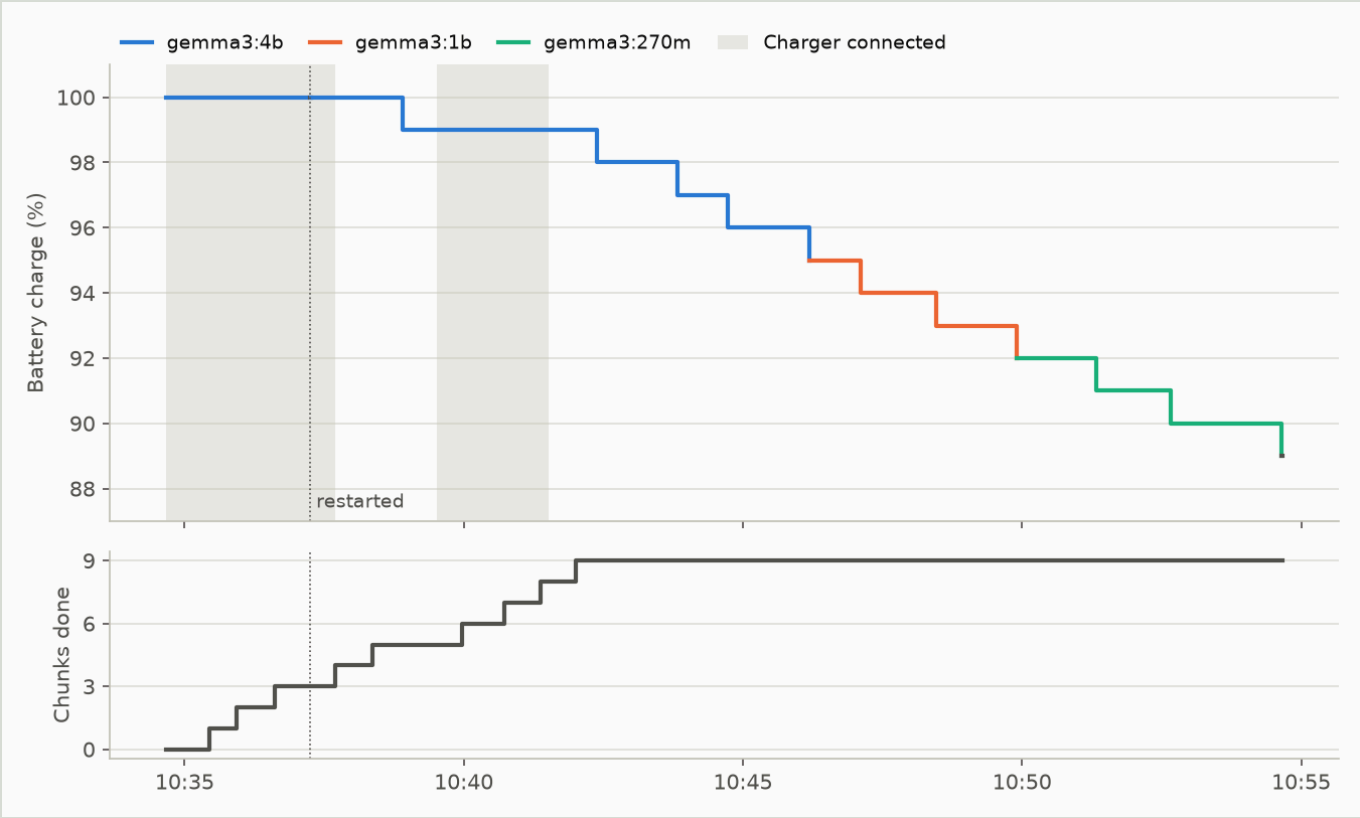
Demonstration, 27 October 2026

Run on a 16 GB laptop (Intel Core i7-1185G7, running models on its processor, 42.8 Wh battery) with Ollama 0.40.5 and Gemma 3 at 4B, 1B and 270M, using the

narrowed bands above. Every step behaved as expected, and all five pass criteria were met.

STEP	WHAT HAPPENED	RESULT
1	Started at 10:34:40 with the charger connected and the battery at 100%. The 4B model loaded, the batch job started, and three chunks finished by 10:36:37, about 30 seconds each.	Pass
2	The process was killed at 10:37:05, 15 seconds into the fourth chunk. On restart at 10:37:15 it reported 269 of 272 chunks left and redid the fourth chunk. The first three were intact and no partial file was left.	Pass
3	The charger was pulled while the fifth chunk was running. The runtime finished that chunk at 10:38:22 and paused the batch job one second later. Test prompts were still answered.	Pass
4	The charger was reconnected and the batch job resumed at the next power check, 10:39:31. Chunks six to eight finished by 10:41:23.	Pass
5	The charger was pulled again while the ninth chunk was running. The runtime finished that chunk at 10:42:01, with the battery at 99%, and paused the batch job one second later. It then switched from 4B to 1B at 95% (10:46:12) and from 1B to 270M at 92% (10:49:54). At 89% (10:54:39) it unloaded the model and exited cleanly.	Pass

CHECK	MEASURED	RESULT
Ladder steps	All three transitions happened on the first reading below each band. Readings were taken every 5 seconds in this run.	Pass
Surplus window	Both times the charger was pulled, the batch job paused one second after the running chunk finished. It resumed at the first check after the charger was reconnected.	Pass
Abrupt stops	9 of 9 finished chunks on disk, numbered in order, none empty, none duplicated, and no temporary files. No errors were logged.	Pass
Reserve	The model was unloaded and the runtime exited at 89%.	Pass
Measurements	Reported in the table below the chart.	Done



Battery charge during the demonstration, coloured by the model in use. Shaded areas are when the charger was connected. The dotted line marks the restart after the process was killed in the middle of a chunk. The lower panel shows finished batch chunks. The repository's plot command draws this chart from the published log.

MODEL	SPEED (TOKENS/S)	AVERAGE DRAW ON BATTERY (W)	TIME IN BAND
4B	7.4	24.8	4.1 min
1B	17.8	23.4	3.6 min
270M	62.3	18.6	4.6 min

From the final drain on battery, 99% to 89% in 12.7 minutes. Speed is the mean of 6, 6 and 8 test prompts. Draw is the laptop's own discharge reading, averaged by its firmware, for the whole laptop including the screen and other open programs. It is not a metered figure.

What the run showed

- **The runtime's decisions worked.** It stepped down, paused, resumed and stopped as designed, and no work was lost across an abrupt stop and two charger pulls.

- **Smaller models were much faster.** On this processor, 1B ran about 2.4 times and 270M about 8 times as fast as 4B.
- **Power draw changed less than speed.** Average draw fell from 24.8 W on 4B to 18.6 W on 270M. A laptop's fixed load, including its screen, makes up much of its draw, and the test prompts were short. The full node's 120 W compute load should make the difference larger, and the pilots will measure it.
- **The 270M model is too weak for real answers.** In a check the same morning it answered "A battery-powered computer is more efficient and durable than a battery-powered one." It can serve as a last rung for short replies and diagnostics only. On the full node, the smallest rung is about 4B.
- **The runtime survived its model server restarting.** In a trial run earlier the same morning, the Ollama server restarted mid-run after a settings change. The runtime logged the error, retried and carried on without losing work.

Next step

A full drain with the default bands, from a full battery down to reserve, to measure each model's speed and power draw over hours rather than minutes, and a screen recording of the demonstration. The repository's README gives the commands for the drain.

Deliverables

A public code repository with the runtime, a README with the steps to reproduce this test, and the log, finished chunks and chart from the demonstration. The code is released under Apache 2.0, and the log, chart and this document under CC BY 4.0.

From laptop to node

The runtime's logic carries over unchanged. In WP1 and WP2, the battery reading moves from the laptop to the charge controller's data port, the surplus test gains a solar forecast, the ladder moves to models of about 4B, 20B and 120B parameters, and checkpoints extend from single chunks to long jobs such as fine-tuning.